



Politechnika Łódzka

Instytut Automatyki

Zakład sterowania robotów

TWORZENIE SYSTEMÓW ROBOTYCZNYCH

Temat 3

ROS2 Services

10 marca 2025



Spis treści

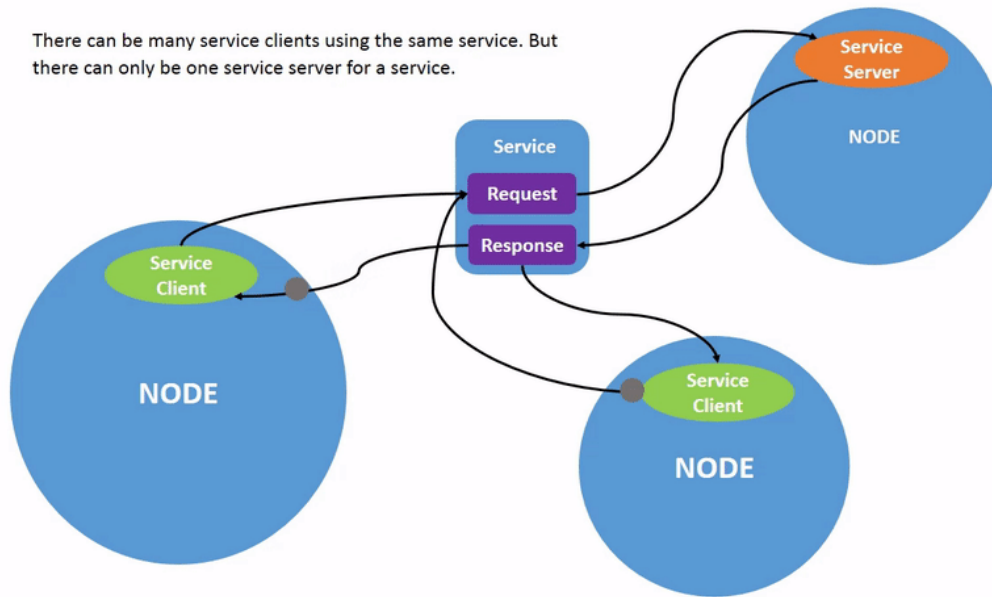
1	Wprowadzenie	2
1.1	Przykład - Włączenie lub wyłączenie światła	2
1.2	Przykład - Pobranie temperatury z czujnika	3
1.3	Przykład - Odczyt aktualnej prognozy pogody	3
1.4	Inne cechy ROS Services	3
2	Struktura wiadomości serwisowej	4
2.1	Gotowe wiadomości ROS2	4
2.2	Jak sprawdzić typ wiadomości serwisowej w ROS2?	4
2.3	Procedura dodania nowej wiadomości serwisowej .srv do pakietu	5
2.3.1	Przejsć do pakietu <code>example_tsr_msgs</code>	5
2.3.2	Utworzenie podkatalogu <code>srv</code>	5
2.3.3	Utworzenie pliku z wiadomością serwisową .srv	5
2.3.4	Dodanie interfejsu do <code>CMakeLists.txt</code>	5
2.3.5	Modyfikacja <code>package.xml</code>	6
2.3.6	Budowanie pakietu	7
2.3.7	Weryfikacja działania	7
3	Procedura tworzenia nowego serwera serwisowego	8
3.1	Przejsć do lokalizacji pakietu <code>example_tsr_project</code>	8
3.2	Utworzenie pliku <code>service_server.py</code>	8
3.3	Ustalenie typu wiadomości	8
3.4	Napisanie kodu dla węzła	9
3.5	Aktualizacja <code>setup.py</code>	10
3.6	Budowanie paczki	10
3.7	Weryfikacja działania	10
4	Procedura dodania nowego klienta serwisowego	11
4.1	Przejsć do lokalizacji pakietu <code>example_tsr_project</code>	11
4.2	Utworzenie pliku <code>service_client.py</code>	12
4.3	Sprawdzenie typu wiadomości obsługiwanej przez serwer serwisowy	12
4.4	Napisanie kodu dla węzła	13
4.5	Aktualizacja <code>setup.py</code>	13
4.6	Budowanie paczki	14
4.7	Weryfikacja działania	14
5	Procedura dodania węzła serwisowego sterującego ruchem robota turtlesim do zadanego punktu	15
5.1	Dodanie wiadomości serwisowej do pakietu <code>example_tsr_msgs</code>	15

5.1.1	Dodanie pliku z wiadomością serwisową.	15
5.1.2	Edycja pliku <code>CMakeLists.txt</code>	15
5.1.3	Edycja <code>package.xml</code>	16
5.1.4	Kompilacja pakietu w workspace.	16
5.1.5	Sprawdzenie poprawności dodania wiadomości	17
5.2	Tworzenie węzła serwisowego w pakiecie <code>example_tsr_project</code> od autonomicznego dojazdu do zadanego punktu	17
5.2.1	Tworzenie pliku <code>turtle_controller.py</code>	17
5.2.2	Rejestracja serwera w <code>setup.py</code>	18
5.2.3	Dodanie brakujących zależności do <code>package.xml</code>	19
5.2.4	Kompilacja pakietu.	19
5.3	Tworzenie węzła klienta w pakiecie <code>example_tsr_project</code>	19
5.3.1	Należy utworzyć plik wykonywalny <code>turtle_client.py</code>	19
5.3.2	Oraz dodać wpis w <code>setup.py</code>	20
5.3.3	Sprawdź serwis	20
5.3.4	Uruchom klienta	20
6	Dokumentacja i inne linki	20

1 Wprowadzenie

ROS Services jest kolejnym sposobem wymiany informacji między węzłami. Jest to mechanizm umożliwiający jednorazową wymianę informacji między klientem a serwerem.

Komunikacja w ROS2 za pomocą Serwisów opiera się na modelu **Request-Response** (żądanie-odpowiedź). Oznacza to, że klient wysyłający żądanie z jednego węzła oczekuje na uzyskanie odpowiedzi od serwera umieszczonego w innym węźle.



Rys. 1.1: Komunikacja dla serwisów. Animacja dostępna na <https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html>

Elementy składowe ROS2 Services:

- Serwer serwisu (service server) – udostępnia określoną usługę. Nasłuchuje na pojawienie się żądania i odpowiada na nie.
- Klient serwisu (service client) – wysyła żądanie do serwera i czeka na odpowiedź.
- Temat serwisu (service names) - unikalna nazwa kanału komunikacyjnego umożliwiającą klientowi wysłanie wiadomości do serwera. Na temacie klient wysyła typ wiadomości zgodny z typem obsługiwanym przez serwer.
- Żądanie (Request) - wiadomość wysyłana przez klienta
- Odpowiedź (Response) – wiadomość zwracana przez serwer.

1.1 Przykład - Włączenie lub wyłączenie światła

Sterownik systemu inteligentnego oświetlenia udostępnia usługę umożliwiającą zmianę stanu światła. Aplikacja sterująca inteligentnym domem wysyła żądanie zmiany stanu światła. Sterownik systemu oświetlenia odbiera żądanie i wykonuje operację.

- Temat serwisu - Protokół umożliwiający wymianę informacji pomiędzy Klientem a Serwerem.
- Klient serwisu - Aplikacja sterująca inteligentnym domem

- Serwer serwisu - Sterownik systemu
- Żądanie - Wiadomość wysyłana przez klienta zawierająca informacje o tym, czy światło ma zostać włączone czy wyłączone.
- Odpowiedź - wysyłana jest przez serwer i informuje o tym czy światło zostało włączone lub wyłączone.

1.2 Przykład - Pobranie temperatury z czujnika

System zarządzania klimatem udostępnia usługę pozwalającą na pobranie aktualnej temperatury otoczenia. System kontroli wentylacji pyta czujnik o aktualną temperaturę, aby dostosować działanie klimatyzacji. Czujnik temperatury odpowiada bieżącą wartością pomiaru.

- Temat serwisu - Protokół umożliwiający wymianę informacji pomiędzy Klientem a Serwerem.
- Klient serwisu - System kontroli wentylacji
- Serwer serwisu - Czujnik temperatury
- Żądanie - Wiadomość wysyłana przez klienta z prośbą o podanie bieżącej temperatury.
- Odpowiedź - Wiadomość zwracana przez serwer zawierająca aktualny odczyt temperatury w stopniach Celsjusza.

1.3 Przykład - Odczyt aktualnej prognozy pogody

System pogodowy udostępnia usługę pozwalającą na pobranie bieżącej prognozy pogody dla wybranej lokalizacji. Aplikacja pogodowa wysyła zapytanie na serwer o prognozę pogody. Centralny serwer meteorologiczny odbiera zapytanie, pobiera dane z czujników pogodowych i zwraca prognozę.

- Temat serwisu - Protokół umożliwiający wymianę informacji pomiędzy Klientem a Serwerem.
- Klient serwisu - Aplikacja pogodowa
- Serwer serwisu - Centralny serwer meteorologiczny
- Żądanie - Wiadomość wysyłana przez klienta z prośbą o podanie prognozy pogody.
- Odpowiedź - Wiadomość zwracana przez serwer zawierająca aktualny odczyt prognozy pogody.

1.4 Inne cechy ROS Services

- Wymagają natychmiastowej odpowiedzi.
- Mają jednorazowy charakter (po zakończeniu operacji nie trzeba monitorować dalszego przebiegu).

Serwisy nie są odpowiednie dla działań długotrwałych, które wymagają aktualizacji statusu (np. jazda robota do celu, skanowanie obszaru), ponieważ do takich zadań lepiej nadają się akcje (actions).

2 Struktura wiadomości serwisowej

Dla komunikacji ROS2 Services dane wysyłane są na unikalną nazwę serwisu pomiędzy węzłami w formie wiadomości o określonej strukturze. Każda wiadomość posiada predefiniowane pola i typy danych, zdefiniowane w plikach `.srv`. Wykorzystywane typy pól w wiadomości są takie same jak dla ROS Topic. Różnica pomiędzy wiadomościami dla ROS Topic a ROS Services związana jest z konstrukcją wiadomości.

Struktura wiadomości serwisowej (.srv) Plik `.srv` składa się z dwóch sekcji:

- Żądanie (Request) – dane wysyłane przez klienta do serwera.
- Oddzielenie sekcji oddzielającej strukturę żądania od odpowiedzi oznaczone jest linią zawierającą trzy myślniki `---`.
- Odpowiedź (Response) – dane zwracane przez serwer do klienta.

Przykład wiadomości serwisowej do pobierania aktualnej temperatury dla podanej lokalizacji.

```
1 string location # Nazwa miasta lub regionu
2 ---
3 float32 temperature # Aktualna temperatura w stopniach Celsjusza
4 float32 humidity # Wilgotność powietrza w procentach
5 string condition # Opis warunków pogodowych (np. "Słonecznie", "Deszczowo")
```

Przykładowe wywołanie serwisu w ROS2

Użytkownik wysyła zapytanie o pogodę w Warszawie (Request):

```
1 location: "Warsaw"
```

Serwer zwraca aktualne warunki pogodowe (Response):

```
1 temperature: 15.5
2 humidity: 60.0
3 condition: "Pochmurno"
```

2.1 Gotowe wiadomości ROS2

ROS2 posiada wiele wbudowanych wiadomości np.:

- `std_srvs` – Empty, SetBool, Trigger,
- `turtlesim` – Kill, SetPen, Spawn, TeleportAbsolute, TeleportRelative,
- `nav2_msgs` – IsPathValid, SetInitialPose, LoadMap,

2.2 Jak sprawdzić typ wiadomości serwisowej w ROS2?

W terminalu można zobaczyć serwisy używane w systemie - lista dostępnych serwisów:

```
1 ros2 service list
```

Sprawdzenie typu wiadomości serwisowej dla serwisu o podanej nazwie `/example_service`:

```
1 ros2 service type /example_service
```

Dla otrzymanego typu sprawdzić strukturę wiadomości typu `Trigger` z pakietu `std_srvs`. Podana wartość jako ostatni argument jest wynikiem otrzymanego wcześniej typu.

```
1  ros2 interface show std_srvs/srv/Trigger
```

Przykładowy wynik:

```
1  ---
2  bool success    # indicate successful run of triggered service
3  string message  # informational, e.g. for error messages
```

W tym przypadku brak pól przed `---` jest prawidłowy i oznacza, że klient nie wysyła żadnych danych. Ważniejsze jest wysłanie sygnału wyzwalającego i uzyskanie odpowiedzi z jego realizacji.

2.3 Procedura dodania nowej wiadomości serwisowej .srv do pakietu

Jest to tylko przypomnienie z wprowadzenia. Podane pliki i katalogi istnieją już w tym pakiecie `example_tsr_msgs`. Procedura natomiast zostanie tak opisana jakby ich nie było. Zakładamy, że dodajemy wiadomości do już istniejącego pakietu, który może nie być skonfigurowany do generowania nowych wiadomości. Może brakować folderów.

2.3.1 Przejść do pakietu `example_tsr_msgs`

Należy w terminalu przejść do workspace'a (`example_tsr_ws`) a następnie do `src` i pakietu `example_tsr_msgs`:

```
1  cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_msgs
```

2.3.2 Utworzenie podkatalogu `srv`

Utworzenie w pakiecie `example_tsr_msgs` katalogów dla interfejsów komunikacyjnych dla ROS Services (katalog `srv`)

W tym celu należy przejść do lokalizacji głównego katalogu dla pakietu a następnie utworzyć katalog `srv`. Można dodać ręcznie katalog z pominięciem terminala.

```
1  cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_msgs
2  mkdir srv
```

2.3.3 Utworzenie pliku z wiadomością serwisową .srv

W katalogu `srv` (`~/tsr_workspaces/example_tsr_ws/src/example_tsr_msgs/srv`) utworzyć plik tekstowy `ExampleSrvType.srv`, w którym zdefiniowany zostanie interfejs komunikacyjny dla wiadomości ROS Services. Wszystkie pliki w tym katalogu będą miały rozszerzenie `.srv`. Do pliku należy wkleić przykład struktury wiadomości:

```
1  int32 request_value
2  ---
3  bool success
4  string message
```

2.3.4 Dodanie interfejsu do `CMakeLists.txt`.

W głównym katalogu pakietu `example_tsr_ws_msgs` znajduje się plik `CMakeLists.txt`, który należy otworzyć w celu edycji.

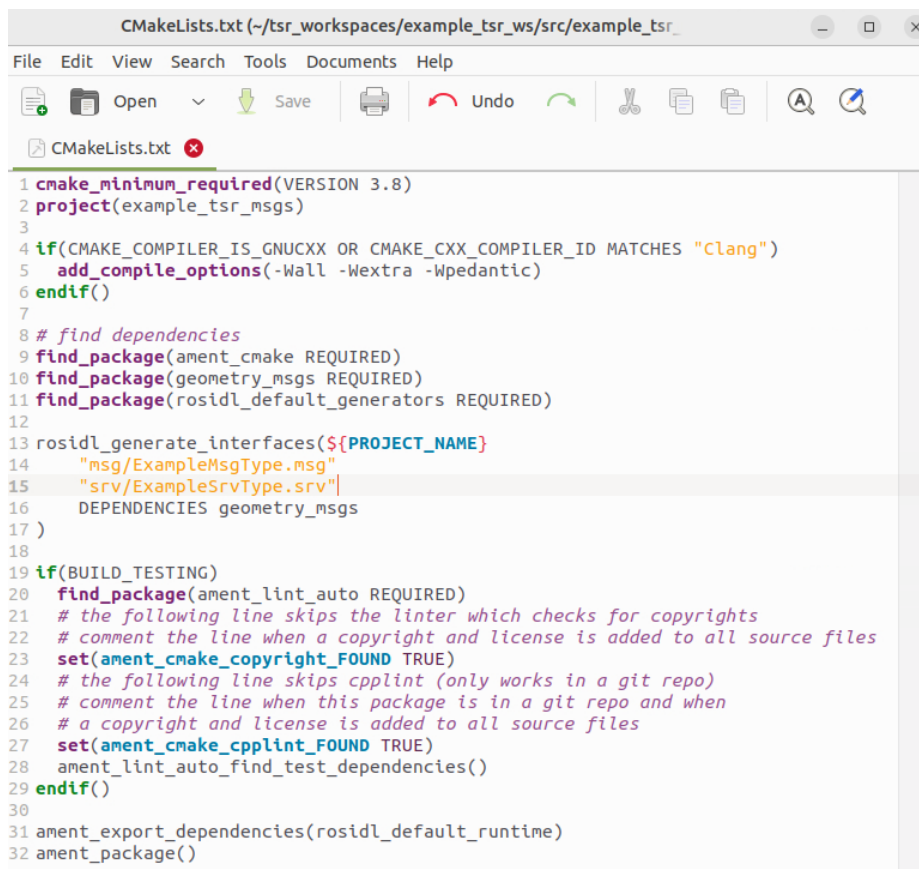
Do pliku należy dodać następujące linie, jeśli brakują:

```
1 find_package(rosidl_default_generators REQUIRED)
2
3 rosidl_generate_interfaces(${PROJECT_NAME}
4 "srv/ExampleSrvType.srv"
5 )
6
7 ament_export_dependencies(rosidl_default_runtime)
```

Do wygenerowania nowych interfejsów komunikacyjnych niezbędne jest zaimportowanie pakietu `rosidl_default_generators` (linia 1).

W części `rosidl_generate_interfaces` (linie 3-5) należy dodać utworzone interfejsy komunikacyjne oraz zależności z nimi związane. Należy również wyeksportować zależności od `message runtime` (linia 7)

Po edycji plik powinien wyglądać następująco (Rys. 2.1):



Rys. 2.1: Plik `CMakeLists.txt` po edycji dla paczki `example_tsr_msgs`

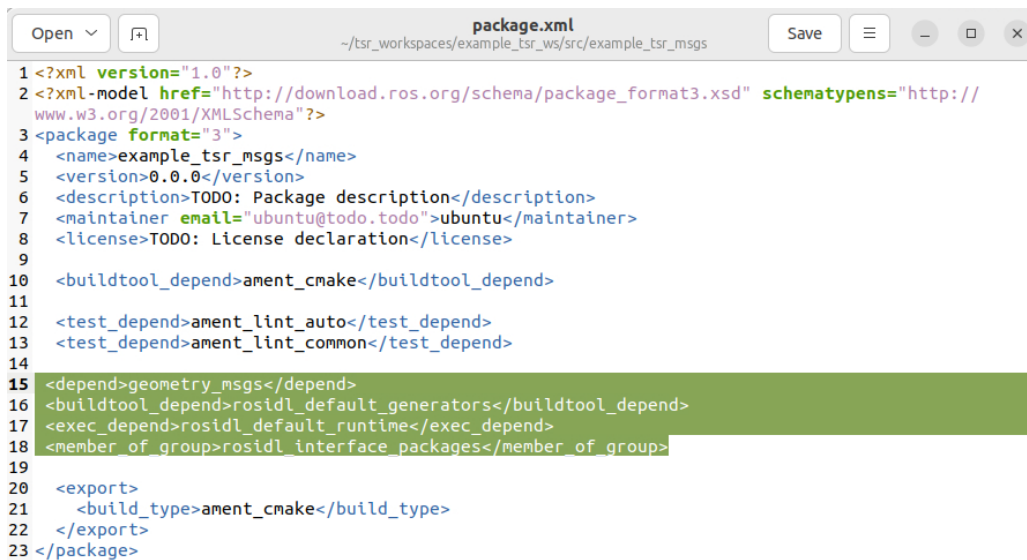
2.3.5 Modyfikacja `package.xml`

Następnie należy dokonać edycji pliku `package.xml` i dodać następujące zależności, jeśli nie są dodane.

```
1 <buildtool_depend>rosidl_default_generators</buildtool_depend>
2 <exec_depend>rosidl_default_runtime</exec_depend>
3 <member_of_group>rosidl_interface_packages</member_of_group>
```

Linia 1 określa narzędzie budowania potrzebne do generowania kodu dla interfejsów komunikacyjnych (`msg`, `srv`, `action`). Linia 2 jest niezbędna do późniejszego wykorzystania interfejsów. `rosidl_interface_packages` jest nazwą grupy z zależnościami w utworzonym pakiecie.

Po edycji plik powinien wyglądać następująco (Rys. 2.2):



Rys. 2.2: Plik package.xml po edycji dla pakietu example_tsr_msgs

2.3.6 Budowanie pakietu

Należy przejść do głównego katalogu dla workspace'a (example_tsr_ws):

```
1 cd ~/tsr_workspaces/example_tsr_ws/src
```

następnie zbudować pojedynczy pakiet (można też zbudować cały workspace colcon build):

```
1 colcon build --packages-select example_tsr_msgs
```

i wykonać ponowny source w terminalu (lub uruchomić nowy terminal):

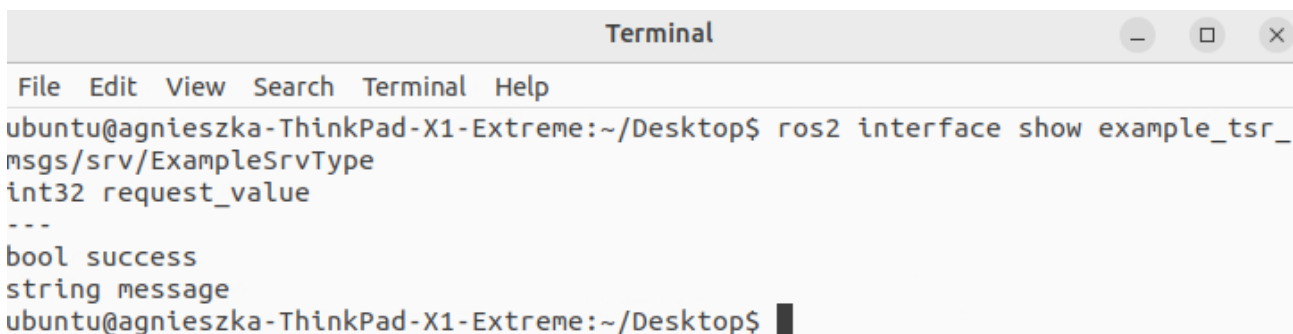
```
1 source install/setup.bash
```

2.3.7 Weryfikacja działania

Dla utworzonego interfejsu dla ROS Services należy sprawdzić czy nowo utworzony interfejs jest widoczny.

```
1 ros2 interface show example_tsr_msgs/srv/ExampleSrvType
```

Prawidłowy rezultat przedstawia (Rys. 2.3):



Rys. 2.3: Wynik operacji sprawdzenia w terminalu struktury utworzonych interfejsów dla pakietu example_tsr_msgs.

Po użyciu polecenia sprawdzającego strukturę interfejsu nie mogą pojawić się błędy. Struktura ma być zgodna z tą jaka jest w dodanym pliku `ExampleSrvType.srv`. Jeśli wyświetla się coś innego tzn., że należy ponownie zweryfikować poprawność konfiguracji i przejść procedurę jeszcze raz zwracając dokładniejszą uwagę na wykonywane kroki.

3 Procedura tworzenia nowego serwera serwisowego

Załóżmy, że mamy już istniejący pakiet (`example_tsr_project`) ROS2 kompilowany jako paczka Python i chcemy do niego dodać serwer serwisowy, korzystając z wiadomości serwisowej `ExampleSrvType` z pakietu `example_tsr_msgs`.

3.1 Przejście do lokalizacji pakietu `example_tsr_project`

Przejdź do pakietu, w którym ma zostać utworzony nowy Serwer serwisowy. W naszym przypadku będzie to pakiet z plikami projektowymi `example_tsr_project`.

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
```

- `~` - oznacza lokalizację katalogu domowego
- `example_tsr_ws` - utworzony workspace. W katalogu `src` znajdują się wszystkie pakiety (nowe lub pobrane np. z githuba).

3.2 Utworzenie pliku `service_server.py`

W pakiecie `example_tsr_project` należy utworzyć nowy plik `service_server.py` w katalogu `example_tsr_project` (w kolejnym katalogu o tej samej nazwie a nie w głównym pakiecie gdzie znajdują się pliki `package.xml` i `setup.py`) a następnie nadać uprawnienia do wykonania. Brak uprawnień jest jednym z występujących problemów z widocznością węzła co uniemożliwia jego uruchomienie pomimo poprawnej konfiguracji paczki.

Należy wpisać kolejne polecenia w terminalu (należy znajdować się w głównej lokalizacji pakietu `example_tsr_project`)

```
1 touch example_tsr_project/service_server.py
2 chmod +x example_tsr_project/service_server.py
```

Nowo utworzony węzeł z Serwerem serwisowym powinien mieć następującą lokalizację:

```
1 ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project/example_tsr_project/service_server.py
```

3.3 Ustalenie typu wiadomości

Następnie należy utworzyć nowy węzeł z Serwerem serwisowym, z którym komunikujemy się poprzez podanie nazwy (`/example_service`) serwera serwisowego. W tym przykładzie do komunikacji zastosowano wiadomość

serwisową typu `ExampleSrvType` z utworzonego pakietu `example_tsr_msgs`. Do utworzenia Serwera serwisowego niezbędna jest znajomość interfejsu komunikacyjnego (struktury przesyłanych wiadomości), który można sprawdzić w terminalu w następujący sposób:

```
1 ros2 interface show example_tsr_msgs/srv/ExampleSrvType
```

Podobnie jak dla ROS Topic możemy w tym miejscu zastosować dowolny interfejs, ale później należy konsekwentnie wykonać zamianę nazw pakietu, z którego pochodzi wiadomość, zamianę typu i struktury obsługiwanej wiadomości na właściwą. W przypadku braku odpowiedniego interfejsu należy go utworzyć.

W wyniku operacji wyświetla się struktura wiadomości dla ROS Service (Rys. 3.1):

```

Terminal
File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 interface show example_tsr_
msgs/srv/ExampleSrvType
int32 request_value
---
bool success
string message
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$

```

Rys. 3.1: Struktura wiadomości serwisowej typu `ExampleSrvType`

Znajomość typów i pól w wiadomości jest niezbędna do jej prawidłowego uzupełnienia.

3.4 Napisanie kodu dla węzła

Następnie należy napisać prosty węzeł z Serwerem serwisowym (zalecane skopiowanie kodu z dołączonych plików lub ich wstawienie, gdyż zachowują wcięcia przy kopiowaniu):

```

1 import rclpy # import biblioteki do obsługi ROS2
2 from rclpy.node import Node
3 from example_tsr_msgs.srv import ExampleSrvType # Import wiadomości serwisowej ROS2 typu ExampleSrvType z
  ↳ pakietu example_tsr_msgs
4
5 class ExampleServiceServer(Node): # Utworzenie klasy dla Node'a
6     def __init__(self):
7         super().__init__('example_service_server') # unikalna nazwa node'a
8
9         # Utworzenie atrybutu klasy będącego Serwerem serwisowym obsługującym wiadomość typu ExampleSrvType
10        ↳ na serwisie o podanej nazwie /example.service. Po otrzymaniu zapytania wywoływana jest natychmiast
11        ↳ metoda self.service_callback
12        self.srv = self.create_service(ExampleSrvType, 'example_service', self.service_callback)
13        self.get_logger().info('Serwis example_service uruchomiony.')
14
15    def service_callback(self, request, response):
16        # request - zapytanie wysłane przez klienta. Dostępne pola wynikają ze struktury wiadomości, które
17        ↳ znajdują się nad ---
18        # response - odpowiedź wysłana przez serwer dla klienta. Dostępne pola wynikają ze struktury
19        ↳ wiadomości, które znajdują się pod ---
20        self.get_logger().info(f'Przetwarzanie zapytania: {request.request_value}')
21
22        # Logika przetwarzania zapytania
23        if request.request_value > 0:
24            response.success = True
25            response.message = "Otrzymano wartość dodatnią."
26        else:
27            response.success = False
28            response.message = "Wartość musi być większa od zera."
29
30        # Zwrócenie odpowiedzi
31        return response
32
33    def main(args=None):
34        # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
35        rclpy.init()
36
37        # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy ExampleServiceServer
38        node = ExampleServiceServer()
39
40        # Spin umożliwia węzłowi ROS2 działanie i oczekuje na zdarzenia. Brak linii spowoduje natychmiastowe
41        ↳ zakończenie działania programu.

```

```
37     rclpy.spin(node)
38
39     # Usuwa węzeł przed zamknięciem programu. Zwalnia zasoby pamięci zajmowane przez ten obiekt.
40     node.destroy_node()
41
42     # Zamyka połączenie z ROS2.
43     rclpy.shutdown()
44
45 if __name__ == '__main__':
46     # uruchomienie node'a
47     main()
48
```

3.5 Aktualizacja *setup.py*

W pakiecie *example_tsr_project* w pliku *setup.py*. Należy zmodyfikować linię z *entry_points* tak, aby dodać nowy węzeł odpowiedzialny za uruchomienie serwera serwisowego, który został przed chwilą utworzony. W tym miejscu należy dodawać uruchomienie każdego nowego węzła (node).

```
1 entry_points={
2     'console_scripts': [
3         'service_server = example_tsr_project.service_server:main',
4     ],
5 }
```

Lewa strona w linii z dodanym **service_server** odnosi się do unikalnej nazwy w pakiecie, która będzie wykorzystywana podczas uruchamiania nowego węzła. Ta nazwa może nie być widoczna po uruchomieniu węzła na liście uruchomionych węzłów (`ros2 node list`), jeśli w kodzie programu jest ona inna. Nazwa węzła wpisana jest w konstruktorze tam gdzie pojawia się `super().__init__('example_service_server')`. Z tego powodu w zaprezentowanym przykładzie nazwą węzła będzie `example_service_server`.

Z prawej strony znajduje się odniesienie do lokalizacji pliku i miejsca w kodzie, z którego uruchamiany jest nowy węzeł. W tym przypadku jest to plik `service_server.py` znajdujący się w katalogu `example_tsr_project`. Uruchomienie następuje w `main`.

3.6 Budowanie paczki

Należy przejść do głównego katalogu dla workspace'a (`example_tsr_ws`):

```
1 cd ~/tsr_workspaces/example_tsr_ws/src
```

następnie zbudować pojedynczą paczkę (można też zbudować cały workspace):

```
1 colcon build --symlink-install --packages-select example_tsr_project
```

i wykonać ponowny `source` w terminalu (lub uruchomić nowy terminal):

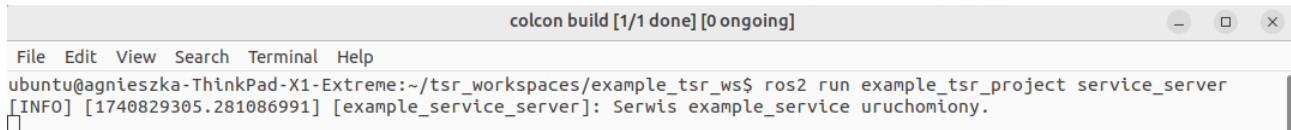
```
1 source install/setup.bash
```

3.7 Weryfikacja działania

W terminalu należy wpisać:

```
1 ros2 run example_tsr_project service_server
```

W wyniku (Rys. 3.2) działania polecenia utworzony został Serwer serwisowy o nazwie `example_service` obsługujący wiadomości typu `ExampleSrvType`. Ponadto w terminalu wyświetla się informacja wysyłana do logów informująca o uruchomieniu serwera.



```
colcon build [1/1 done] [0 ongoing]
File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ ros2 run example_tsr_project service_server
[INFO] [1740829305.281086991] [example_service_server]: Serwis example_service uruchomiony.
```

Rys. 3.2: Wynik uruchomienia nowego węzła `example_service_server`

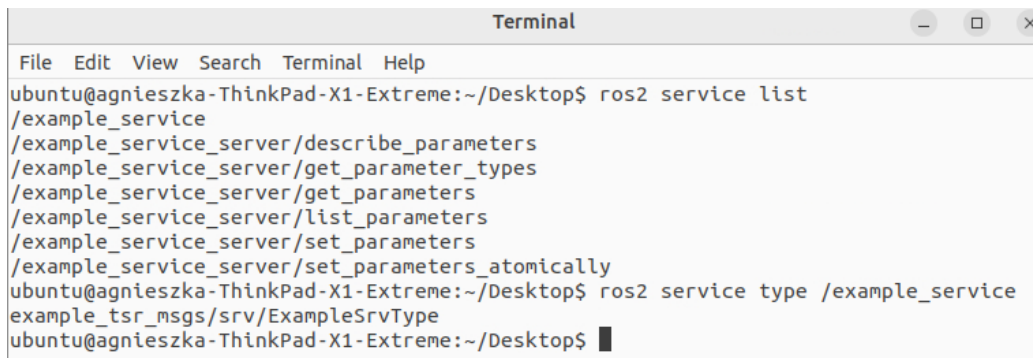
W nowym terminalu można sprawdzić listę aktywnych serwisów i na liście powinien pojawić się `example_service`:

```
1 ros2 service list
```

a następnie komendę umożliwiającą podgląd typu wiadomości obsługiwanej przez serwer o nazwie `/example_service`.

```
1 ros2 service type /example_service
```

W wyniku wpisanych komend w terminalu powinien być wyświetlony następujący rezultat (Rys. 3.3):



```
Terminal
File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 service list
/example_service
/example_service_server/describe_parameters
/example_service_server/get_parameter_types
/example_service_server/get_parameters
/example_service_server/list_parameters
/example_service_server/set_parameters
/example_service_server/set_parameters_atomically
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 service type /example_service
example_tsr_msgs/srv/ExampleSrvType
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$
```

Rys. 3.3: Sprawdzenie poprawności działania węzła `example_service_server`

4 Procedura dodania nowego klienta serwisowego

Załóżmy, że mamy już istniejący pakiet (`example_tsr_project`) ROS2 kompilowany jako paczka Python i chcemy do niego dodać klienta serwisowego wysyłającego wiadomość serwisową typu `ExampleSrvType` z pakietu `example_tsr_msgs`.

4.1 Przejście do lokalizacji pakietu `example_tsr_project`

Przejdź do pakietu, w którym ma zostać utworzony nowy klient serwisowy. W naszym przypadku będzie to pakiet z plikami projektowymi `example_tsr_project`.

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
```

- ~ - oznacza lokalizację katalogu domowego
- `example_tsr_ws` - utworzony workspace. W katalogu `src` znajdują się wszystkie pakiety (nowe lub pobrane np. z githuba).

4.2 Utworzenie pliku `service_client.py`

W pakiecie `example_tsr_project` należy utworzyć nowy plik `service_client.py` w katalogu `example_tsr_project` (w kolejnym katalogu o tej samej nazwie a nie w głównym pakiecie gdzie znajdują się pliki `package.xml` i `setup.py`) a następnie nadać uprawnienia do wykonania. Brak uprawnień jest jednym z występujących problemów z widocznością węzła co uniemożliwia jego uruchomienie pomimo poprawnej konfiguracji paczki.

Należy wpisać kolejne polecenia w terminalu (przed wpisaniem poleceń lokalizacja w terminalu powinna odnosić się do głównej lokalizacji pakietu `example_tsr_project`)

```
1 touch example_tsr_project/service_client.py
2 chmod +x example_tsr_project/service_client.py
```

Nowo utworzony węzeł z klientem serwisowym powinien mieć następującą lokalizację:

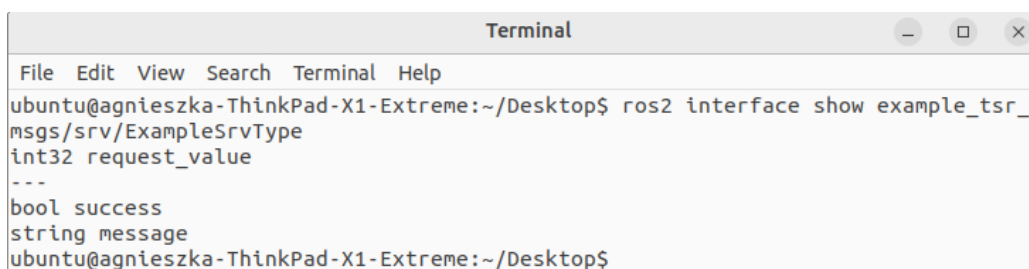
```
1 ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project/example_tsr_project/service_client.py
```

4.3 Sprawdzenie typu wiadomości obsługiwanej przez serwer serwisowy

Następnie należy utworzyć nowy węzeł z klientem serwisowym, z którym komunikujemy się poprzez podanie nazwy (`/example_service`) serwera serwisowego. W tym przykładzie do komunikacji zastosowano wiadomość serwisową typu `ExampleSrvType` z utworzonego pakietu `example_tsr_msgs`. Do utworzenia klienta serwisowego niezbędna jest znajomość interfejsu komunikacyjnego dla serwera.

```
1 ros2 interface show example_tsr_msgs/srv/ExampleSrvType
```

W wyniku operacji wyświetla się struktura wiadomości dla ROS Service (Rys. 4.1):



Rys. 4.1: Struktura wiadomości serwisowej typu `ExampleSrvType`

Znajomość typów i nazw pól w wiadomości jest niezbędna do jej prawidłowego uzupełnienia.

4.4 Napisanie kodu dla węzła

Następnie należy napisać prosty węzeł z Klientem serwisowym (zalecane skopiowanie kodu z dołączonych plików lub ich wstawienie, gdyż zachowują wcięcia przy kopiowaniu):

```

1  import rclpy # import biblioteki do obsługi ROS2
2  from rclpy.node import Node
3  from example_tsr_msgs.srv import ExampleSrvType # Import wiadomości serwisowej ROS2 typu ExampleSrvType z
   ↪ pakietu example_tsr_msgs
4
5
6  class ExampleServiceClient(Node): # Utworzenie klasy dla Node'a
7      def __init__(self):
8          super().__init__('example_service_client') # unikalna nazwa node'a
9
10         # Utworzenie atrybutu klasy będącego Klientem serwisowym wysyłającym żądanie typu ExampleSrvType na
   ↪ serwisie o podanej nazwie /example_service. Nie następuje jeszcze wystanie.
11         self.client = self.create_client(ExampleSrvType, 'example_service')
12
13         # Pętla oczekująca na dostępność serwisu. Jeśli nie jest jeszcze uruchomiony to zapobiega wystąpienia
   ↪ błędu wynikającego z jego braku.
14         while not self.client.wait_for_service(timeout_sec=1.0):
15             self.get_logger().info('Oczekiwanie na dostępność serwisu...')
16
17     def send_request(self, value): # metoda od wysyłania żądania na serwer. value jest dowolną przekazywaną
   ↪ wartością typu int
18         # Utworzenie żądania. Zwrócić uwagę na pojawiający się po . Request()
19         request = ExampleSrvType.Request()
20
21         # Uzupełnienie pola wiadomości przekazaną w metodzie klasy wartością value.
22         request.request_value = value
23
24         self.get_logger().info(f'Wysyłanie żądania z wartością: {value}')
25
26         # Wywołuje serwis asynchronicznie (bez blokowania programu), wysyłając request do serwera. Zwraca
   ↪ obiekt future, który reprezentuje przyszłą odpowiedź od serwera.
27         future = self.client.call_async(request)
28
29         # Gwarantuje, że program nie przejdzie dalej, dopóki serwer nie zwróci odpowiedzi.
30         rclpy.spin_until_future_complete(self, future)
31
32         # Pobiera wynik z future, czyli faktyczną odpowiedź zwróconą przez serwer.
33         response = future.result()
34         if response:
35             self.get_logger().info(f'Odpowiedź serwera: success={response.success},
   ↪ message="{response.message}"')
36         else:
37             self.get_logger().error('Błąd podczas komunikacji z serwerem.')
38
39
40     def main():
41         # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
42         rclpy.init()
43
44         # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy ExampleServiceClient
45         client_node = ExampleServiceClient()
46
47         # Wysłanie żądania na serwer /example_service
48         client_node.send_request(10)
49
50         # Usuwa węzeł przed zamknięciem programu. Zwalnia zasoby pamięci zajmowane przez ten obiekt.
51         client_node.destroy_node()
52
53         # Zamyka połączenie z ROS2.
54         rclpy.shutdown()
55
56
57     if __name__ == '__main__':
58         # uruchomienie node'a
59         main()
60
61

```

4.5 Aktualizacja *setup.py*

W pakiecie *example_tsr_project* w pliku *setup.py*. Należy zmodyfikować linię z *entry_points* tak, aby dodać nowy węzeł odpowiedzialny za uruchomienie klienta serwisowego, który został przed chwilą utworzony. W tym miejscu należy dodawać uruchomienie każdego nowego węzła (node).

```

1 entry_points={
2     'console_scripts': [
3         'service_client = example_tsr_project.service_client:main',
4     ],
5 }

```

Lewa strona w linii z dodanym `service_client` odnosi się do unikalnej nazwy w pakiecie, która będzie wykorzystywana podczas uruchamiania nowego węzła. Ta nazwa może nie być widoczna po uruchomieniu węzła na liście uruchomionych węzłów (`ros2 node list`), jeśli w kodzie programu jest ona inna. Nazwa węzła wpisana jest w konstruktorze tam gdzie pojawia się `super().__init__('example_service_client')`. Z tego powodu w zaprezentowanym przykładzie nazwą węzła będzie `example_service_client`.

Z prawej strony znajduje się odniesienie do lokalizacji pliku i miejsca w kodzie, z którego uruchamiany jest nowy węzeł. W tym przypadku jest to plik `service_client.py` znajdujący się w katalogu `example_tsr_project`. Uruchomienie następuje w `main`.

4.6 Budowanie paczki

Należy przejść do głównego katalogu dla workspace'a (`example_tsr_ws`):

```

1 cd ~/tsr_workspaces/example_tsr_ws/src

```

następnie zbudować pojedynczą paczkę (można też zbudować cały workspace):

```

1 colcon build --symlink-install --packages-select example_tsr_project

```

i wykonać ponowny `source` w terminalu (lub uruchomić nowy terminal):

```

1 source install/setup.bash

```

4.7 Weryfikacja działania

W terminalu należy wpisać:

```

1 ros2 run example_tsr_project service_client

```

W wyniku (Rys. 4.2) działania polecenia utworzony został węzeł z klientem serwisowym wysyłający żądanie na serwer o nazwie `example_service` obsługujący wiadomości typu `ExampleSrvType`. Ponadto w terminalu wyświetla się informacja wysyłana do logów informująca o wysłaniu żądania na serwer. Warunkiem prawidłowego działania jest pozostawienie uruchomionego w poprzednim rozdziale nowego serwera serwisowego.

```

colcon build [1/1 done] [0 ongoing]
File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ ros2 run example_tsr_project service_client
[INFO] [1740832720.510252808] [example_service_client]: Wysyłanie żądania z wartością: 10
[INFO] [1740832720.512383163] [example_service_client]: Odpowiedź serwera: success=True, message="Otrzymano wartość dodatnią."
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$

```

Rys. 4.2: Wynik uruchomienia nowego węzła `example_service_client`

5 Procedura dodania węzła serwisowego sterującego ruchem robota turtlesim do zadanego punktu

W tym rozdziale utworzymy nowy serwer serwisowy, który będzie dodawał nowy cel dojazdu dla robota `turtle1` w symulacji `turtlesim`. Natomiast cały węzeł, w którym znajduje się serwer będzie odpowiadał za dojazd do kolejnych dodanych punktów. Na podstawie aktualnego położenia robota będzie wysyłał odpowiednie prędkości sterujące. Dla serwisu zostanie utworzony specjalny interfejs serwisowy umożliwiający przesłanie informacji o nowym zleconym punkcie do dojazdu. Dodatkowo na temacie `/turtle1/status` jest publikowana informacja o statusie związanym z jazdą:

- FREE - wolny, brak punktów na liście lub wszystkie zostały osiągnięte
- IN PROGRESS - w trakcie jazdy
- DONE - wyświetla się zaraz po osiągnięciu punktu. Następnie pojawia się status FREE lub IN PROGRESS zależnie od tego czy wszystkie punkty zostały osiągnięte lub nie

5.1 Dodanie wiadomości serwisowej do pakietu `example_tsr_msgs`

5.1.1 Dodanie pliku z wiadomością serwisową.

W pakiecie `example_tsr_msgs` katalogu `example_tsr_msgs/srv/` utwórz plik `AddGoToPose.srv` i dodaj do niego:

```
1 turtlesim/Pose goal_pose # Współrzędne docelowe
2 ---
3 bool success # Czy dodano nowy cel dla robota
4 string message # Komunikat o wyniku
```

Wiadomość `AddGoToPose.srv` zawiera:

- Żądanie (request) – współrzędne celu (`Pose` z pakietu `turtlesim`).
- Odpowiedź (response) – informację o powodzeniu (`success`) oraz komunikat (`message`).

5.1.2 Edycja pliku `CMakeLists.txt`

Dodaj do sekcji `find_package` zależność:

```
1 find_package(turtlesim REQUIRED)
```

A następnie do `generate_interfaces`:

```
1 rosidl_generate_interfaces(${PROJECT_NAME}
2 "srv/AddGoToPose.srv"
3 DEPENDENCIES turtlesim
4 )
```

```

1 cmake_minimum_required(VERSION 3.8)
2 project(example_tsr_msgs)
3
4 if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
5   add_compile_options(-Wall -Wextra -Wpedantic)
6 endif()
7
8 # find dependencies
9 find_package(ament_cmake REQUIRED)
10 find_package(geometry_msgs REQUIRED)
11 find_package(turtlesim REQUIRED)
12 find_package(rosidl_default_generators REQUIRED)
13
14 rosidl_generate_interfaces(${PROJECT_NAME}
15   "msg/ExampleMsgType.msg"
16   "srv/ExampleSrvType.srv"
17   "srv/AddGoToPose.srv"
18   DEPENDENCIES geometry_msgs turtlesim
19 )
20
21 if(BUILD_TESTING)
22   find_package(ament_lint_auto REQUIRED)
23   # the following line skips the linter which checks for copyrights
24   # comment the line when a copyright and license is added to all source files
25   set(ament_cmake_copyright_FOUND TRUE)
26   # the following line skips cpplint (only works in a git repo)
27   # comment the line when this package is in a git repo and when
28   # a copyright and license is added to all source files
29   set(ament_cmake_cpplint_FOUND TRUE)
30   ament_lint_auto_find_test_dependencies()
31 endif()
32
33 ament_export_dependencies(rosidl_default_runtime)
34 ament_package()

```

Rys. 5.1: Plik CMakeLists.txt po modyfikacji

5.1.3 Edycja package.xml

Dodaj zależność:

```
1 <depend>turtlesim</depend>
```

```

1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3 <package format="3">
4   <name>example_tsr_msgs</name>
5   <version>0.0.0</version>
6   <description>TODO: Package description</description>
7   <maintainer email="ubuntu@todo.todo">ubuntu</maintainer>
8   <license>TODO: License declaration</license>
9
10  <buildtool_depend>ament_cmake</buildtool_depend>
11
12  <test_depend>ament_lint_auto</test_depend>
13  <test_depend>ament_lint_common</test_depend>
14
15  <depend>geometry_msgs</depend>
16  <depend>turtlesim</depend>
17  <buildtool_depend>rosidl_default_generators</buildtool_depend>
18  <exec_depend>rosidl_default_runtime</exec_depend>
19  <member_of_group>rosidl_interface_packages</member_of_group>
20
21  <export>
22    <build_type>ament_cmake</build_type>
23  </export>
24 </package>

```

Rys. 5.2: Plik package.xml po modyfikacji

5.1.4 Kompilacja pakietu w workspace.

Otworzyć terminal z poziomu widoku workspace lub przejść do niego po otwarciu nowego terminala:

```
1 cd ~/tsr_workspaces/example_tsr_ws/
```

Następnie skompilować pakiet.

```
1 colcon build --packages-select example_tsr_msgs
2 source install/setup.bash
```

5.1.5 Sprawdzenie poprawności dodania wiadomości

W terminalu wpisać

```
1 ros2 interface show example_tsr_msgs/srv/AddGoToPose
```

W wyniku (Rys. 5.3) powinna pojawić się struktura wiadomości:

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 interface show example_tsr_msgs/srv/AddGoToPose
turtlesim/Pose goal_pose # Współrzędne docelowe
    float32 x
    float32 y
    float32 theta
    float32 linear_velocity
    float32 angular_velocity
---
bool success # Czy dodano nowy cel dla robota
string message # Komunikat o wyniku
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ █
```

Rys. 5.3: Sprawdzenie poprawności dodania AddGoToPose.srv

5.2 Tworzenie węzła serwisowego w pakiecie example_tsr_project od autonomicznego dojazdu do zadanego punktu

5.2.1 Tworzenie pliku turtle_controller.py

W pakiecie example_tsr_project katalogu example_tsr_project utwórz plik turtle_controller.py i dodaj do niego:

```
1 import rclpy
2 import math
3 from rclpy.node import Node
4 from geometry_msgs.msg import Twist
5 from std_msgs.msg import String
6 from turtlesim.msg import Pose
7 from example_tsr_msgs.srv import AddGoToPose
8
9
10 class TurtleController(Node):
11     def __init__(self):
12         super().__init__('turtle_controller') # unikalna nazwa node'a
13         # Serwis
14         self.srv = self.create_service(AddGoToPose, 'turtle_go_to_pose', self.add_go_to_pose_callback)
15
16         # Publikowanie prędkości
17         self.cmd_vel_publisher = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
18
19         # Publikowanie statusu
20         self.status_publisher = self.create_publisher(String, '/turtle1/status', 10)
21
22         # Subskrypcja pozycji
23         self.pose_subscription = self.create_subscription(Pose, '/turtle1/pose', self.pose_callback, 10)
24
25         # Aktualna pozycja
26         self.current_pose = Pose()
27         # Aktualny status
28         self.current_status = String()
29
30         # Lista aktualnych celów do których ma dojechać robot.
31         self.goals = []
32         # Utworzenie timera, co 1s wywołuje się metoda self.timer_callback
33         self.timer = self.create_timer(0.1, self.movement_callback)
34         self.get_logger().info("Serwer turtle_go_to_pose uruchomiony.")
35
36     def pose_callback(self, msg):
37         """Aktualizacja bieżącej pozycji robota"""
```

```

38     self.current_pose = msg
39
40 def add_go_to_pose_callback(self, request, response):
41     """Obsługa żądania serwisowego - sterowanie robotem do celu"""
42     self.get_logger().info(f'Otrzymano żądanie: X={request.goal_pose.x}, Y={request.goal_pose.y}')
43     self.goals.append(request.goal_pose)
44     response.success = True
45     response.message = "Dodano nowy cel."
46     self.get_logger().info(response.message)
47     return response
48
49 def movement_callback(self):
50     if self.goals:
51         vel_msg = Twist()
52         if self.euclidean_distance(self.goals[0]) >= 0.1:
53             # Porportional controller.
54             # https://en.wikipedia.org/wiki/Proportional_control
55
56             if abs(self.angular_vel(self.goals[0])) > 0.2:
57                 # Angular velocity in the z-axis.
58                 vel_msg.angular.z = 0.6
59             else:
60                 # Linear velocity in the x-axis.
61                 vel_msg.linear.x = self.linear_vel(self.goals[0])
62                 # Angular velocity in the z-axis.
63                 vel_msg.angular.z = self.angular_vel(self.goals[0])
64
65             # Publishing our vel_msg
66             self.cmd_vel_publisher.publish(vel_msg)
67             self.current_status.data = "IN PROGRESS"
68             self.status_publisher.publish(self.current_status)
69         else:
70             vel_msg.linear.x = 0.0
71             vel_msg.angular.z = 0.0
72             self.cmd_vel_publisher.publish(vel_msg)
73
74             self.current_status.data = "DONE"
75             self.status_publisher.publish(self.current_status)
76             del self.goals[0]
77         else:
78             self.current_status.data = "FREE"
79             self.status_publisher.publish(self.current_status)
80
81 def euclidean_distance(self, goal_pose):
82     """Euclidean distance between current pose and the goal."""
83     return math.sqrt(math.pow((goal_pose.x - self.current_pose.x), 2) +
84                     math.pow((goal_pose.y - self.current_pose.y), 2))
85
86 def linear_vel(self, goal_pose, constant=1.5):
87     """See video: https://www.youtube.com/watch?v=Qh15No15htM."""
88     vel = constant * self.euclidean_distance(goal_pose)
89     return min(max(-1.5, vel), 1.5)
90
91 def steering_angle(self, goal_pose):
92     """See video: https://www.youtube.com/watch?v=Qh15No15htM."""
93     return math.atan2(goal_pose.y - self.current_pose.y, goal_pose.x - self.current_pose.x)
94
95 def angular_vel(self, goal_pose, constant=6.0):
96     """See video: https://www.youtube.com/watch?v=Qh15No15htM."""
97     return constant * (self.steering_angle(goal_pose) - self.current_pose.theta)
98
99
100 def main():
101     # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
102     rclpy.init()
103     # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy SimplePublisher
104     node = TurtleController()
105     # Spin umożliwia węzłowi ROS2 działanie i oczekuje na zdarzenia. Brak linii spowoduje natychmiastowe
106     # zakończenie działania programu.
107     rclpy.spin(node)
108     # Usuwa węzeł przed zamknięciem programu. Zwalnia zasoby pamięci zajmowane przez ten obiekt.
109     node.destroy_node()
110     # Zamyka połączenie z ROS2.
111     rclpy.shutdown()
112
113 if __name__ == '__main__':
114     main()

```

5.2.2 Rejestracja serwera w setup.py

Otwórz setup.py i dodaj wpis w entry_points:

```

1 entry_points={
2     'console_scripts': [
3         'turtle_controller = example_tsr_project.turtle_controller:main',
4     ],
5 },

```

5.2.3 Dodanie brakujących zależności do package.xml

W pliku package.xml zależność od pakietu turtlesim została dodana wcześniej więc nie trzeba jej już dodawać. Brakuje natomiast zależności od std_msgs, którą należy dodać w pliku package.xml.

```
1 <exec_depend>std_msgs</exec_depend>
```

5.2.4 Kompilacja pakietu.

Otworzyć terminal z poziomu widoku workspace lub przejść do niego po otwarciu nowego terminala:

```
1 cd ~/tsr_workspaces/example_tsr_ws/
```

Następnie skompilować pakiet.

```
1 colcon build --symlink-install --packages-select example_tsr_project
2 source install/setup.bash
```

5.3 Tworzenie węzła klienta w pakiecie example_tsr_project

Węzeł klienta odpowiada za dodanie nowego celu do dojazdu. Prawidłowe wysłanie celu spowoduje dodanie go do listy odwiedzanych punktów co rozpoczęcie autonomicznej jazdy po kolejnych dodanych do listy punktach.

5.3.1 Należy utworzyć plik wykonywalny turtle_client.py

```
1 # Jazda na podstawie https://wiki.ros.org/turtlesim/Tutorials/Go%20to%20Goal
2 import rclpy
3 from rclpy.node import Node
4 from example_tsr_msgs.srv import AddGoToPose
5 from geometry_msgs.msg import Pose
6
7
8
9 class TurtleClient(Node):
10     def __init__(self):
11         super().__init__('turtle_client')
12         self.client = self.create_client(AddGoToPose, 'turtle_go_to_pose')
13
14         while not self.client.wait_for_service(timeout_sec=1.0):
15             self.get_logger().info("Oczekiwanie na dostępność serwisu...")
16
17     def send_goal(self, x, y):
18         request = AddGoToPose.Request()
19         request.goal_pose.x = x
20         request.goal_pose.y = y
21
22         self.get_logger().info(f"Wysyłanie żądania dojazdu do: X={x}, Y={y}")
23         future = self.client.call_async(request)
24         rclpy.spin_until_future_complete(self, future)
25
26         response = future.result()
27         if response:
28             self.get_logger().info(f"Odpowiedź: success={response.success}, message='{response.message}'")
29         else:
30             self.get_logger().error("Brak odpowiedzi z serwera.")
31
32 def main():
33     rclpy.init()
34     client = TurtleClient()
35     client.send_goal(3.0, 3.0) # Przykładowa pozycja docelowa
36     client.destroy_node()
37     rclpy.shutdown()
38
39 if __name__ == '__main__':
40     main()
41
```

5.3.2 Oraz dodać wpis w setup.py

Otwórz `setup.py` i dodaj wpis w `entry_points`:

```
1 entry_points={  
2     'console_script': [  
3         'turtle_client = example_tsr_project.turtle_client:main',  
4     ],  
5 }
```

```
1 ros2 run example_tsr_project turtle_controller
```

5.3.3 Sprawdź serwis

```
1 ros2 service list
```

5.3.4 Uruchom klienta

```
1 ros2 run example_tsr_project turtle_client
```

Po zakończeniu robot powinien rozpocząć dojazd do pozycji.

6 Dokumentacja i inne linki

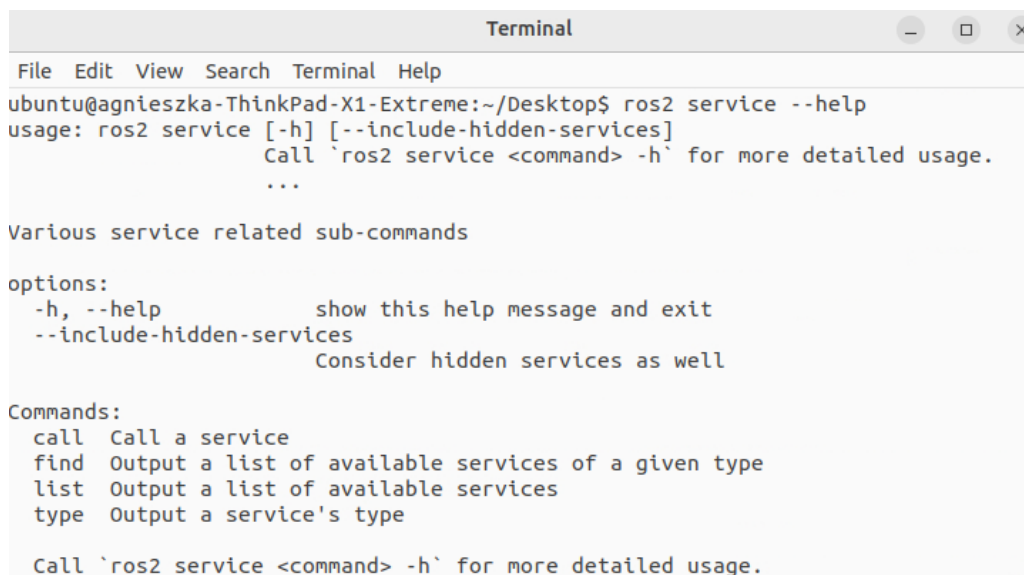
Rozdział może zawierać dokumentację poleceń, linki do źródeł na podstawie, których powstała instrukcja oraz inne linki pozwalające poszerzyć wiedzę.

— Dokumentacja ROS Humble

<https://docs.ros.org/en/humble/index.html>

— Więcej o serwisach

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html>



```
Terminal
File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 service --help
usage: ros2 service [-h] [--include-hidden-services]
                  Call 'ros2 service <command> -h' for more detailed usage.
...

Various service related sub-commands

options:
  -h, --help            show this help message and exit
  --include-hidden-services
                        Consider hidden services as well

Commands:
  call  Call a service
  find  Output a list of available services of a given type
  list  Output a list of available services
  type  Output a service's type

Call 'ros2 service <command> -h' for more detailed usage.
```

Rys. 6.1: Dokumentacja polecenia `ros2 service`